

Lesson 15 — Overfitting & Regularization

We now move from Lesson 14: Model Tuning & Hyperparameter Optimization to Lesson 15: Handling Overfitting & Regularization.

1. What is Overfitting?

Overfitting happens when a model learns the training data too well, including noise and random patterns.

Example:

```
Training Accuracy → 99%
Testing Accuracy  → 72%
```

The model performs extremely well on data it has seen but poorly on new/unseen data.

Think:

```
Training data
  ↓
Model memorizes too much
  ↓
Excellent training performance
  ↓
Poor new-data performance
```

2. Underfitting vs Good Fit vs Overfitting

Underfitting

```
Training → 65%
Testing  → 63%
```

The model is too simple and hasn't learned enough.

Good Fit

```
Training → 92%
Testing  → 90%
```

The model learned useful patterns and generalizes well.

Overfitting

```
Training → 99%
Testing  → 72%
```

The model learned the training data too specifically.

3. Why Does Overfitting Happen?

Common causes:

- Model is too complex
- Too many features
- Too little training data
- Too many training iterations
- Noise in the data
- Tree grows too deep
- Neural network is too large

For example:

```
DecisionTreeClassifier(max_depth=None)
```

could create a very complex tree.

4. How Can We Reduce Overfitting?

Common methods:

```
Reduce model complexity
↓
Regularization
↓
Cross-validation
↓
Feature selection
↓
More training data
↓
Early stopping
```

5. What is Regularization? ☆

Regularization adds a penalty for overly complex models.

The basic idea is:

```
Normal objective
+
Complexity penalty
↓
Better generalization
```

Instead of allowing the model to become unnecessarily complex, regularization encourages it to stay simpler.

6. L1 Regularization

L1 regularization adds a penalty based on the absolute values of coefficients.

Conceptually:

```
Loss +  $\lambda \cdot \sum |w_i|$ 
```

Here:

w = model coefficients λ = regularization strength

In scikit-learn, Lasso uses L1 regularization.

```
from sklearn.linear_model import Lasso

model = Lasso(alpha=0.1)

model.fit(X_train, y_train)
```

A useful property of L1:

It can force some coefficients to become exactly zero.

Therefore, it can also perform a form of feature selection.

7. L2 Regularization

L2 uses the squared coefficients:

$$\text{Loss} + \lambda \cdot \sum w_i^2$$

In scikit-learn:

```
from sklearn.linear_model import Ridge

model = Ridge(alpha=1.0)

model.fit(X_train, y_train)
```

L2 generally makes coefficients smaller, rather than usually forcing them exactly to zero.

8. L1 vs L2

L1 (Lasso)	L2 (Ridge)
Uses $ w $ (absolute value of coefficients)	Uses w^2 (squared coefficients)
Can make coefficients zero	Usually makes coefficients smaller
Useful for feature selection	Useful for controlling complexity
Lasso	Ridge

9. What is alpha?

Consider:

```
Ridge(alpha=1.0)
```

alpha controls the strength of regularization.

Generally:

```
Small alpha
↓
Weak regularization
↓
```

More complex model

while:

```
Large alpha
  ↓
Strong regularization
  ↓
Simpler model
```

But too much regularization can cause underfitting.

10. Regularization and Model Tuning

This connects directly to our previous lesson.

We can tune alpha:

```
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import Ridge

model = Ridge()

param_grid = {
    "alpha": [0.01, 0.1, 1, 10, 100]
}

grid = GridSearchCV(
    model,
    param_grid,
    cv=5,
    scoring="neg_mean_squared_error"
)

grid.fit(X_train, y_train)

print(grid.best_params_)
```

So:

```
Lesson 14
Hyperparameter tuning
  ↓
Find good alpha
  ↓
Lesson 15
Control model complexity
```

11. Decision Tree Regularization

Decision Trees don't use Ridge/Lasso in the same way.

Instead, we control tree complexity with parameters such as:

```
DecisionTreeClassifier(
    max_depth=5,
    min_samples_split=10,
    min_samples_leaf=5
)
```

max_depth

Controls how deep the tree can grow.

```
Small max_depth
  ↓
Simpler tree
  ↓
Less overfitting
```

min_samples_split

Minimum number of samples required to split a node.

min_samples_leaf

Minimum number of samples allowed in a leaf.

These are examples of complexity control.

12. Random Forest and Overfitting

Random Forest generally reduces overfitting compared with a single Decision Tree because it combines many trees.

Important parameters include:

```
RandomForestClassifier(
    n_estimators=100,
    max_depth=5,
    min_samples_leaf=2
)
```

You can tune these using the techniques from Lesson 14.

13. How Do We Detect Overfitting?

One simple method is to compare:

```
Training score
  vs
Validation/Test score
```

Example:

```
train_score = model.score(X_train, y_train)
test_score = model.score(X_test, y_test)

print("Training:", train_score)
print("Testing:", test_score)
```

If you get:

```
Training: 0.99
Testing: 0.70
```

there is a strong sign of overfitting.

14. Cross-Validation Helps

Instead of relying on one split:

Train → Test

we can use:

Train → Cross-validation → Validation scores

This gives us a more reliable estimate of how well the model generalizes.

That's why we learned Cross-Validation in Lesson 14.

15. Bias-Variance Idea ☆

This is an important concept behind overfitting.

High Bias

Model is too simple.

Underfitting

High Variance

Model changes too much based on the training data.

Overfitting

The goal is to find a good balance:

High Bias ← Good Balance → High Variance
Underfit Overfit

16. Complete Simple Example

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error

# Load dataset
data = load_diabetes()

X = data.data
y = data.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# Ridge model
model = Ridge(alpha=1.0)

# Train
model.fit(X_train, y_train)

# Predictions
```

```

y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)

# Evaluate
train_error = mean_squared_error(
    y_train,
    y_train_pred
)

test_error = mean_squared_error(
    y_test,
    y_test_pred
)

print("Training MSE:", train_error)
print("Testing MSE:", test_error)

```

Here we're using Ridge Regression to control model complexity through L2 regularization.

Lesson 15 — What You Must Remember

Overfitting

Model performs very well on training data but poorly on unseen data.

Underfitting

Model is too simple and performs poorly even on training data.

Regularization

Technique used to discourage overly complex models and improve generalization.

```

L1
Lasso
↓
Can make coefficients = 0
↓
Feature selection

```

```

L2
Ridge
↓
Shrinks coefficients
↓
Controls complexity

```

Decision Trees

Use:

- max_depth
- min_samples_split
- min_samples_leaf

to control complexity.

Most important connection

```

Lesson 14
↓
Hyperparameter Tuning
↓
Find good parameters

```

↓
Lesson 15
↓
Control Overfitting
↓
Better Generalization

One-line definition:

Overfitting is when a model learns the training data too specifically, while regularization and complexity control help it generalize better to unseen data.